This is a significant opportunity with Quest Global. The Job Description (JD) signals a role that leans heavily into **Cyber Security** and **DevSecOps**, rather than a pure Backend Engineering role.

To succeed, you must bridge the gap between your **Backend/Architecture experience** and their **Security/Vulnerability Management requirements**.

Here is a strategic preparation plan tailored to pivot your resume experience toward this specific JD.

1. The Strategy: "Secure by Design" Developer

You are not just a Java Developer; you are a generic "Security-Aware Engineer." Since you have the **Cyber Security Foundation Course** and hands-on experience with **JWT/Nginx**, you need to frame your backend work through a security lens.

Your Narrative Pivot:

- **Instead of saying:** "I upgraded Java 8 to 17 for better features."
- **Say:** "I led platform modernization to patch critical security vulnerabilities found in legacy Java versions, ensuring our stack was compliant and secure against modern CVEs."

2. Tackling the "Nessus" & Security Requirement

The JD explicitly mentions **Nessus** (a vulnerability scanner). If you haven't used Nessus directly, do not fake it. Instead, pivot to your experience with **remediation**.

- **What is Nessus?** It scans infrastructure/code for known vulnerabilities (like outdated SSL libraries or unpatched OS versions).
- **Your Answer Strategy:**

"While I haven't configured Nessus scans personally, I have extensive experience acting on vulnerability reports as a **Production Owner**. For example, during my platform modernization project (Java 8 to 17), I systematically eliminated vulnerabilities related to legacy dependencies. I understand the remediation workflow: Scan -> Analyze Severity -> Patch/Mitigate -> Verify."

Technical Concepts to Review:

- **OWASP Top 10:** Be ready to explain how you prevent SQL Injection (you use Hibernate/Spring Data, which handles this), XSS, and Broken Access Control in your Spring Boot APIs.
- **JWT & Auth:** You listed **JWT** on your resume. Be prepared to whiteboard how your authentication flow works. (e.g., "Do you sign the token? What algorithm? Where do you store it?").

- **mTLS/SSL:** Since you worked on microservices, did you use mutual TLS between services? If not, be ready to discuss how you secured service-to-service communication.

3. Java & Scripting Alignment

The JD asks for **Java/C++** and **Linux Shell Scripting**.

Java (Your Strong Suit):

- Focus on **Security features in Java 17**. Mention how stronger encapsulation in newer Java versions aids security.
- **Spring Security:** If you used Spring Boot 3, discuss how you configured SecurityFilterChain beans.

Shell Scripting & Automation:

- You have strong **Docker/Kubernetes** experience. This implies you know Linux.
- **Preparation:** Review basic shell commands for log analysis (grep, awk, sed) and process management (ps, top, kill).
- **The "Automation" Angle:** Cite your **Git** and **CI/CD** experience. Explain how you automated the deployment pipeline to reduce human error (a security risk).

C++ vs. Rust:

- The JD mentions C++, but you know **Rust**.
- **Strategy:** If asked about C++, mention that while your core expertise is Java, your knowledge of **Rust** gives you a strong grasp of memory safety and low-level system handling, which parallels modern C++ best practices.

4. Behavioral Questions (Using Your Projects)

Use your existing achievements to answer behavioral questions, but emphasize the **risk/security** aspect.

JD Requirement	Your Experience (The Pivot)
	Project: Upgrade from Spring Boot 2.x to 3.x.
Analyzing Results & Mitigation	**Narrative:** "I analyzed the breaking changes and security patches required for the upgrade, mitigating the risk of using EOL (End of Life) frameworks."

JD Requirement	Your Experience (The Pivot)
	Project: MongoDB/Neo4j Optimization.
Databases & PL/SQL	**Narrative:** "I optimized queries to prevent resource exhaustion attacks (DoS). I ensured data integrity during the 15k accounts/hour migration." **Award:** Bravo Award/Production Owner.
Handling Incidents	**Narrative:** "I was the point person for critical production issues. I can handle the pressure of a security incident response."

Export to Sheets

5. Interview Logistics Checklist

- **Documents:**
  - [ ] Printed copies of your updated resume (bring at least 3).
  - [ ] Valid Government ID (Aadhar/PAN/Passport) – Security at tech parks is strict.
  - [ ] The "Cyber Security Foundation" certificate (print a copy if you have it; it proves interest).
- **Location:**
  - **Quest Global, Brookefield:** This is in EPIP Zone. Traffic in Brookefield/Whitefield can be unpredictable on Saturdays due to construction or events. Plan to reach by **8:45 AM**.
- **Contact:**
  - Save Santhoshi's and Mr. S. Vidyasagar's numbers in your phone now.

6. Mock Question for Practice

Interviewer: "We use Nessus to find vulnerabilities. How would you handle a report showing 50 critical vulnerabilities in your Java application?"

Suggested Answer: "First, I would prioritize them based on **exploitability**—is this vulnerability actually reachable in our production environment? I would then group

them by dependency. often, upgrading a single parent library (like Spring Boot) resolves multiple CVEs at once. For valid risks that cannot be patched immediately, I would look for mitigation steps, such as applying WAF rules or restricting network access, while we work on the code fix."

Would you like a quick "Cheat Sheet" of common Linux Shell commands used in security logs analysis?

This is a comprehensive, deep-dive preparation guide tailored to bridge your **Backend Software Engineer** profile with the **Cyber Security/DevSecOps** requirements of the Quest Global role.

Use these Q&A sets to practice your "pivot" strategy: acknowledging your developer background while proving you are "security-native."

Section 1: Cyber Security & Vulnerability Management (The Gap Closer)

*The JD explicitly mentions **Nessus**. As a developer, you aren't expected to be a full-time penetration tester, but you **must** understand the remediation workflow.*

Concept: The Vulnerability Lifecycle (Scan -> Analyze -> Remediate)

Q1: We use Nessus for vulnerability scanning. How do you handle a report with 50+ Critical vulnerabilities?

- **The "Analyst" Answer:** "I send it to the developer." (Bad)
- **Your "Production Owner" Answer:** "I treat vulnerability reports as production incidents.
    1. **Triage:** I first group them by **root cause**. Often, 20 vulnerabilities map to a single outdated dependency (e.g., an old Log4j version).
    2. **Verification:** I check if the vulnerable code path is actually exposed in our environment (False Positive check).
    3. **Remediation:** For true positives, I prioritize upgrading the library. If an upgrade breaks the build, I implement a **temporary mitigation** (like a WAF rule or disabling the specific feature) while we refactor the code."

Q2: What is the difference between an Authenticated (Credentialed) vs. Unauthenticated Nessus scan?

- **Answer:** "Unauthenticated scans only see what an outsider sees (open ports, banner grabbing). **Credentialed scans** log into the system (via SSH/Windows) and inspect the internal file system, registry, and patch levels. As a developer, I rely on **Credentialed scans** because they find the 'hidden' vulnerabilities like outdated local libraries (JARs) that aren't listening on a port."

Q3: How do you prevent the "OWASP Top 10" in your Java Microservices?

- **Answer:** (Pick 2-3 specific examples)
    - **Injection (SQLi):** "I use Hibernate/JPA which relies on Parameterized Queries by default, preventing SQL injection."

- **Broken Auth:** "I implemented JWT with short expiration times and refresh tokens. We verify the signature (RS256) on every request in the API Gateway."
- **Sensitive Data Exposure:** "I ensure all PII in the DB is encrypted at rest, and we use TLS 1.3 for all data in transit. In my Spring Boot apps, I use @JsonIgnore to prevent accidentally returning password hashes in API responses."

Section 2: Java 17 & Backend Architecture (Your Strength)

*Quest Global looks for strong core fundamentals. Don't just list features; explain **why** you use them.*

Concept: Modern Java Features

Q4: Why did you move from Java 8/11 to Java 17? What are the security benefits?

- **Answer:** "Beyond performance (G1GC improvements), Java 17 offers **Strong Encapsulation** (Project Jigsaw). It prevents unauthorized access to internal JDK APIs, which reduces the attack surface. We also use **Records** for DTOs to ensure immutability (data can't be tampered with after creation) and **Sealed Classes** to strictly control class inheritance."

Q5: How do you handle Concurrency in a high-load system?

- **Answer:** "In my Data Migration service (15k accounts/hour), I couldn't just spawn unlimited threads. I used a **ThreadPoolExecutor** with a bounded queue to prevent memory exhaustion (OOM). For IO-bound tasks (like calling external APIs), I used **CompletableFuture** to run tasks asynchronously without blocking the main thread."

Q6: How does Spring Security work internally?

- **Answer:** "It's essentially a chain of **Servlet Filters**. When a request hits the app, it passes through the SecurityFilterChain. The UsernamePasswordAuthenticationFilter extracts credentials, passes them to the AuthenticationManager, which validates them against our UserDetailsService. If valid, a SecurityContext is established for that session."

Section 3: Linux Shell & Automation (The "Hands-On" Requirement)

The JD asks for "Linux Shell Scripting" and "Automation."

Concept: Log Analysis & Process Management

Q7: A production server is responding slowly. How do you debug it using only the CLI?

- **Answer:** "I follow a standard checklist:
    1. top or htop: Check for high CPU or Memory usage. (Is it a Java process? Note the PID).
    2. df -h: Check if the disk is full (common cause for crashes).
    3. tail -f /var/log/app.log | grep "ERROR": Watch live logs for exceptions.
    4. netstat -tulnp: Check which ports are open and if there are too many 'WAIT' connections (DDOS or connection leak)."

Q8: Write a script to find all files larger than 100MB and delete them.

- **Answer:** (Be careful—mentioning "delete" without "verify" is risky!)
    - "I would first **list** them to be safe: find /path -type f -size +100M -print
    - "Once verified, I would add the delete flag: find /path -type f -size +100M -delete or use -exec rm {} \;"

Q9: How do you automate a daily backup of your MongoDB?

- **Answer:** "I would write a shell script using mongodump, compress it with tar, and upload it to AWS S3. Then, I'd schedule it using crontab -e.
    - *Example Cron:* 0 2 * * * /scripts/backup_mongo.sh (Runs at 2:00 AM daily)."

Section 4: Behavioral & Quest Global Culture

*Quest Global values **Ownership**, **Ethics**, and **Customer Focus**.*

Q10: Describe a time you had a conflict with a team member over a technical decision.

- **Pivot to Security:** "My peer wanted to disable SSL validation in the Dev environment to speed up testing. I pushed back because 'Dev configs often leak to Prod,' which is a huge security risk. Instead, I spent an hour helping them set up a proper local certificate. This ensured we maintained security standards without blocking their velocity."

Q11: Ethics Question: You find a vulnerability that could let you access user data. No one else knows. What do you do?

- **Answer:** "I would document it immediately and report it to the Security Team/Management. I would **not** exploit it to 'prove' it works. I would then create a Jira ticket with the severity labeled 'Critical' and propose a fix. At my previous job, I won an award for proactively fixing production issues—integrity is part of that ownership."

Quick Recap for the Interview Day

1. **Nessus:** You **Remediate** and **Analyze** reports (Production Owner mindset).
2. **Java:** You use **Java 17** for **Security** and **Immutability**.
3. **Linux:** You use it for **Debugging** and **Automation** (Cron/Grepping logs).
4. **C++:** "I know **Rust**, which handles memory safety even better than C++, so I understand the low-level concepts."

Relevance: This video is excellent for reviewing the most common Linux commands you'll face in the interview, specifically focusing on the grep, find, and process management commands mentioned in the guide above.

Linux Command Line Tutorial For Beginners 40 - grep command